CIS437 - Cloud Computing

– Cloud Design Patterns II –

Patterns for managing service access and failure

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Based on the original version by Professor Erik Fredericks

Last Class

- What Is a Design Pattern?
- Why Cloud Applications Need Design Patterns
- Cloud Fallacies
- Publish/Subscribe Pattern
- Geode Pattern
- Compare the Patterns
- Practice

1. What problem are we trying to solve?
2. What is the key architectural idea?
3. When would we use it?”

Last Class Practice

Scenario

1. A factory has hundreds of sensors continuously producing temperature and equipment-status events. Several applications need to consume those events independently. **Which pattern fits best? Why?**
2. An online service has customers in North America, Europe, and Asia. It needs low latency and should remain available even if one geographic deployment fails. **Which pattern fits best? Why?**
3. A global e-commerce company runs its application in several geographic regions. When a customer places an order, multiple independent services need to process the event—for example, inventory, shipping, and analytics. **Which pattern or patterns could be useful here? Explain what problem each pattern solves.**



Agenda

- Quick Introduction to Distributed Services
- Facade Pattern in Software Engineering
- API Gateway Pattern
- Circuit Breaker Pattern
- Compare the Four Cloud Patterns
- Practice

A Quick Look at Distributed Services

Monolithic application

- Many application functions are deployed together as one application

Distributed services / microservices

- Application functions are separated into independently running services
- Each service focuses on a specific business capability

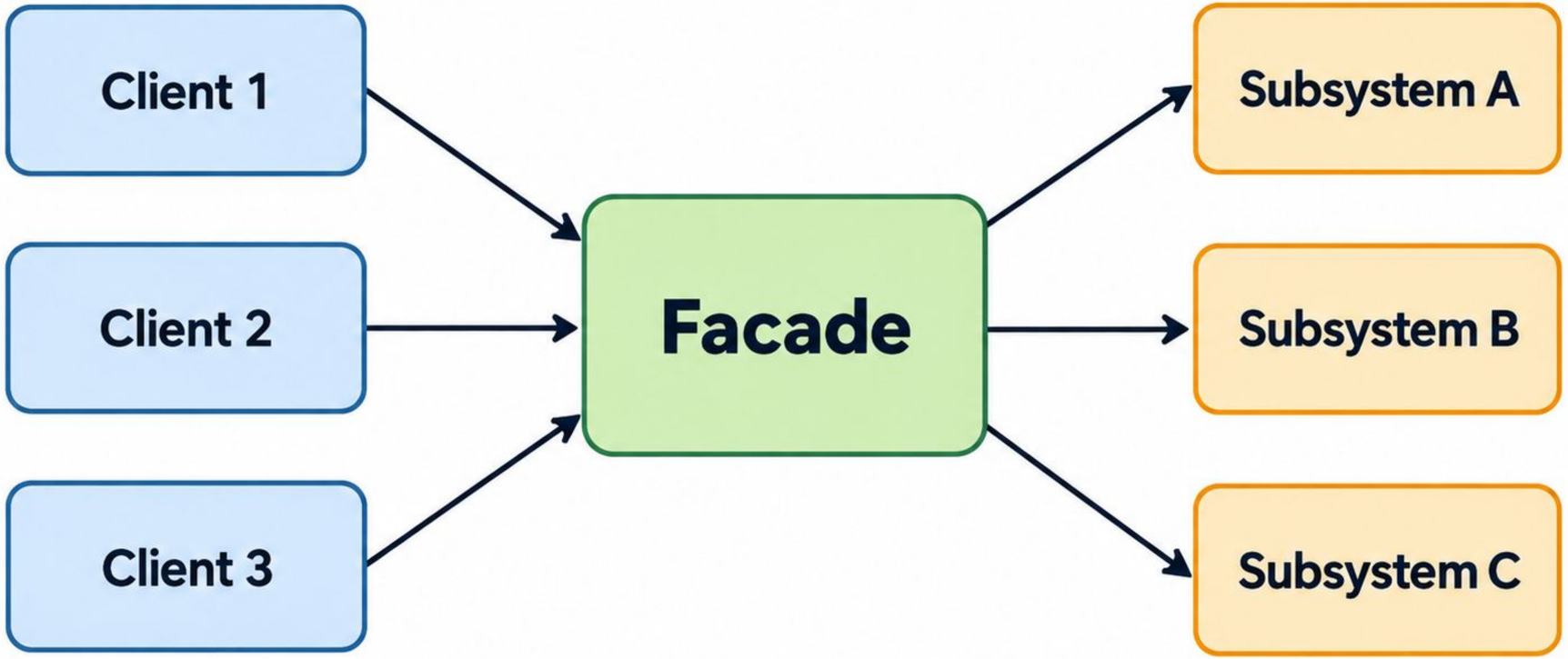
Example:

- Catalog Service
- Order Service
- Payment Service
- User Service



If we have a web application or mobile application, does the mobile app need to know where the Catalog Service is, where the Order Service is, where the Payment Service is, and how to authenticate with every one of them?

Facade Pattern in Software Engineering



Facade Pattern in Software Engineering

- Provides one simple interface to a more complex subsystem
- Hides internal complexity from clients
- Clients do not need to interact directly with every internal component

Key idea: one simple interface in front of many components

Why might this be useful when a system has many internal components?

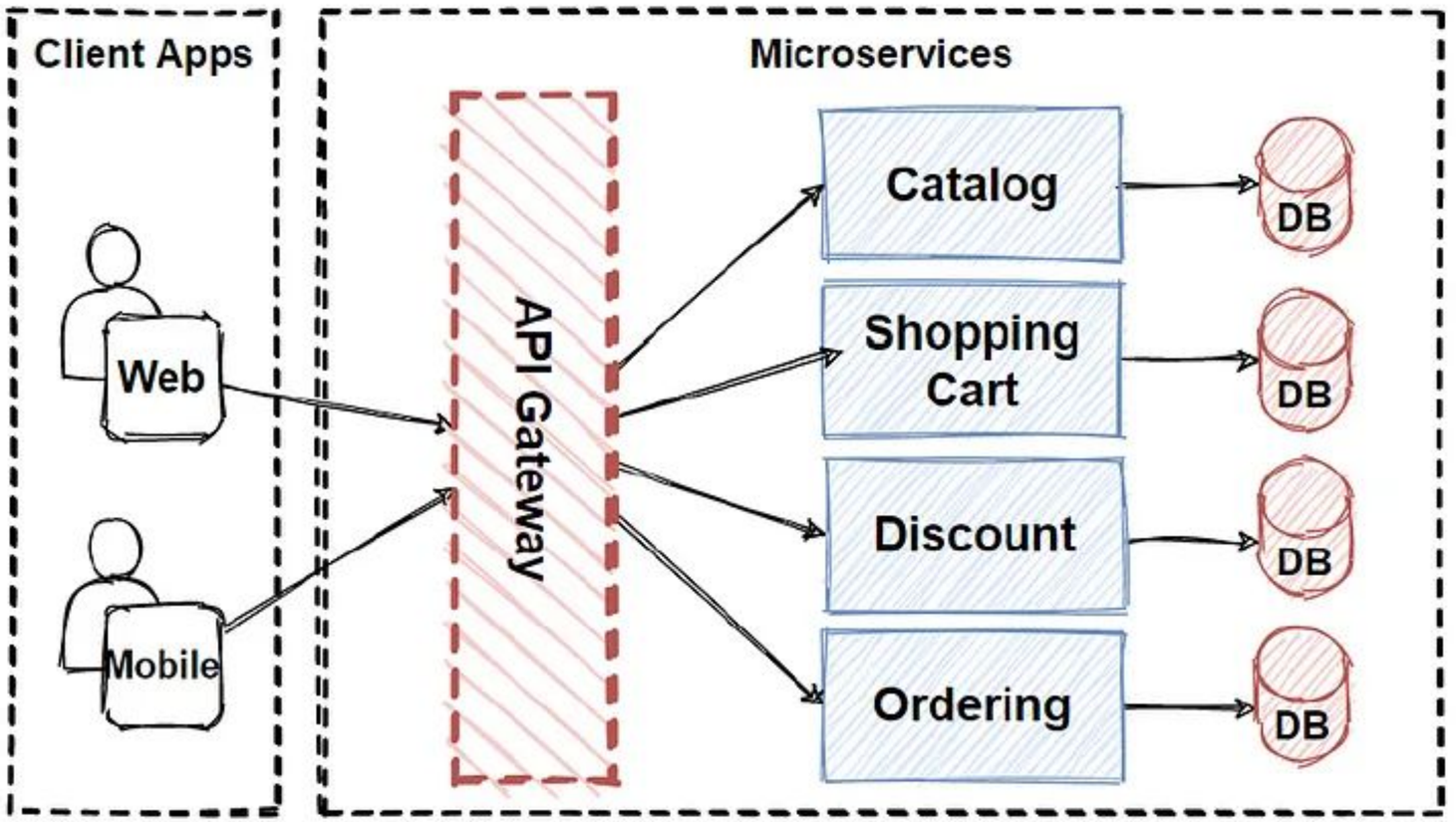


API Gateway

Useful when an application has:

- Multiple backend services
- Multiple client types, such as web, mobile, or partner applications

Similar idea to Facade: one entry point hides backend complexity



API Gateway Pattern

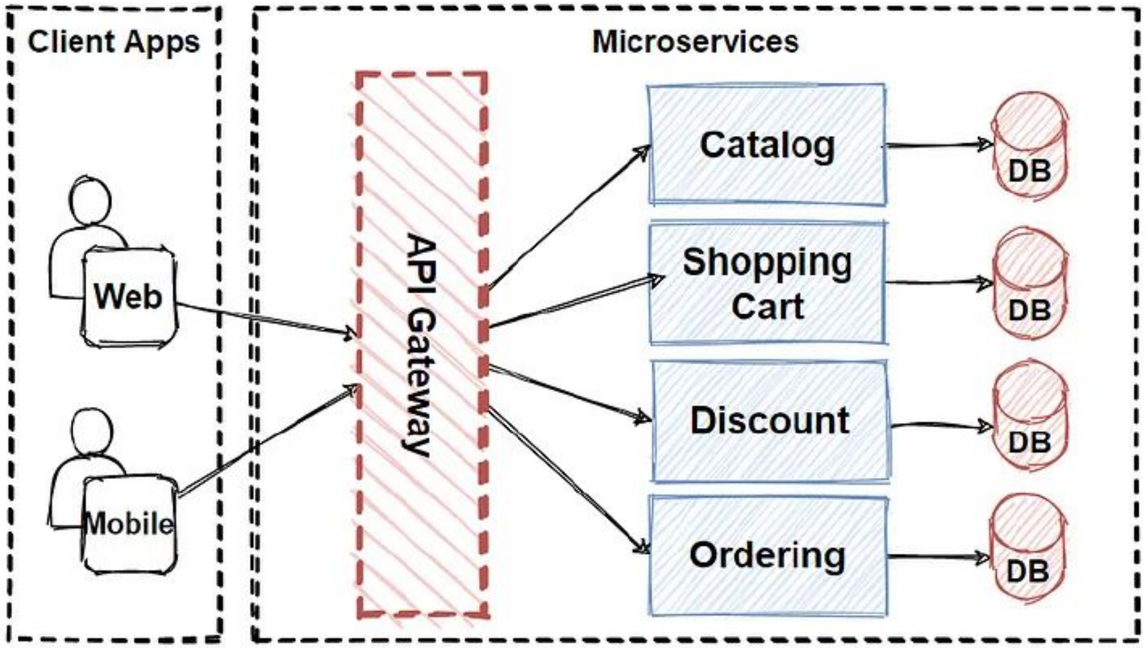
API Gateway

API Gateway

- Provides a single entry point from clients to backend services
- Hides backend complexity
- Routes requests to the appropriate service
- Works across networked/distributed services

Compared with Facade

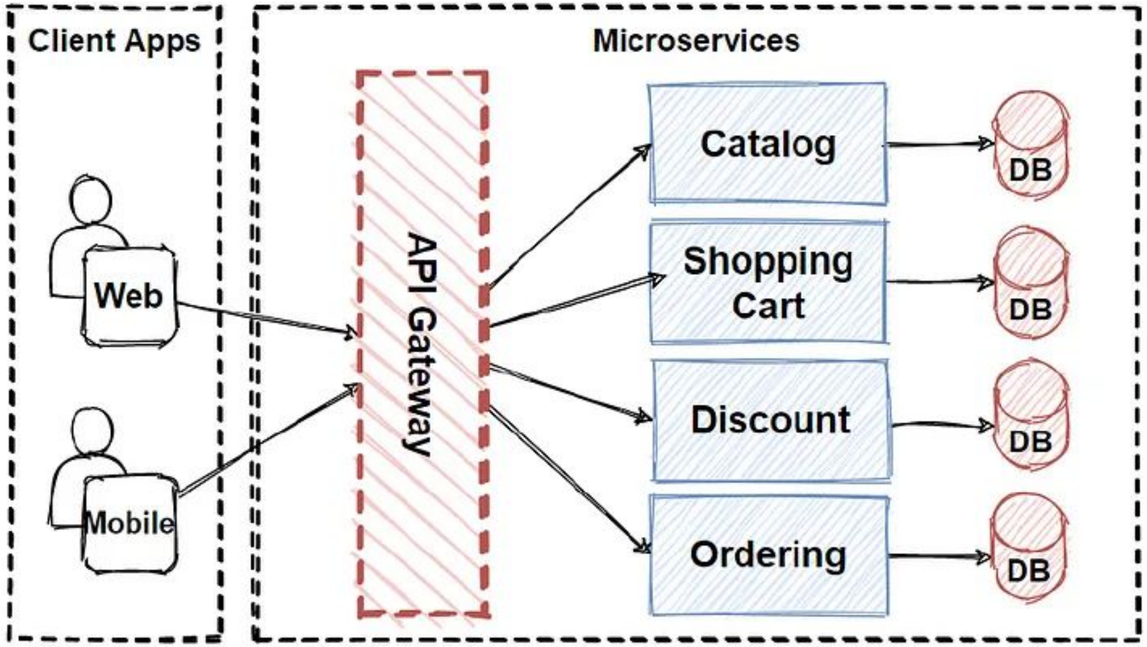
- Facade is a software-design abstraction
- API Gateway actively handles network requests and routing



How could we implement this?

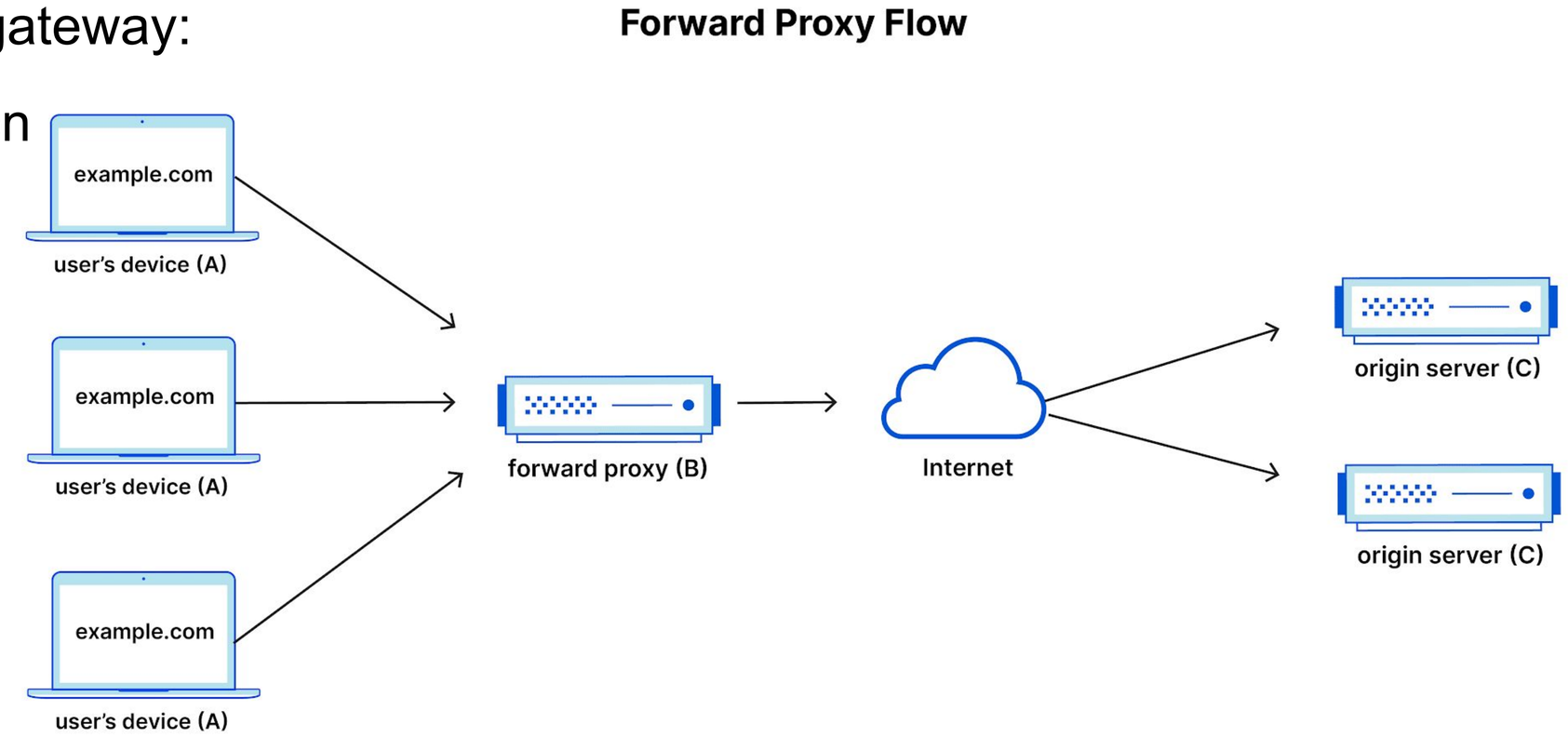


What responsibilities would this component need to handle?



How Could We Implement This?

- Use a **managed API gateway service**
- Use a **reverse proxy / gateway server**
- Define routes from API endpoints to backend services
- Add common controls at the gateway:
  - Authentication / authorization
  - Rate limiting
  - Monitoring / logging



API Gateway: Advantages and Trade-offs

Advantages?

Disadvantages / Trade-offs?



API Gateway

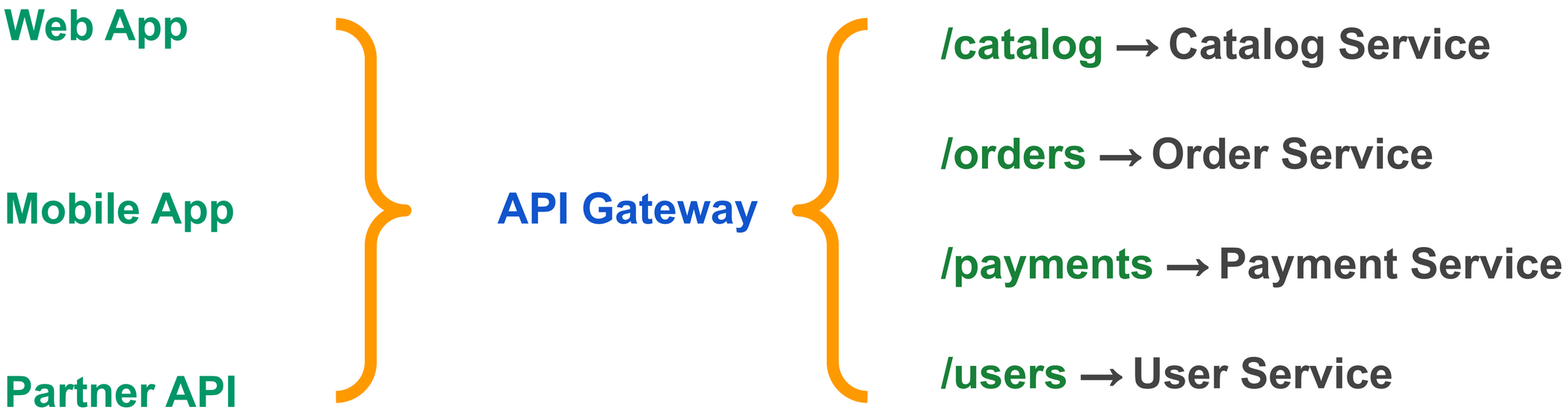
Advantages

- Can aggregate requests from multiple backend services into one response
- Can help route or distribute requests to backend services
- Can centralize authentication and authorization
- Can provide a common place for rate limiting, monitoring, and logging

Disadvantages / Trade-offs

- Can become a critical dependency if not designed for high availability
- Adds another component to the architecture
- Can become a bottleneck
- Putting too much business logic in the gateway increases complexity

API Gateway Example



Clients need to know only the gateway endpoint, not every backend service.

Circuit breaker

Temporarily stops calls to a failing or unhealthy service after repeated failures

Possible causes:

- Network disruption
- Downstream service unavailable
- Downstream service overloaded

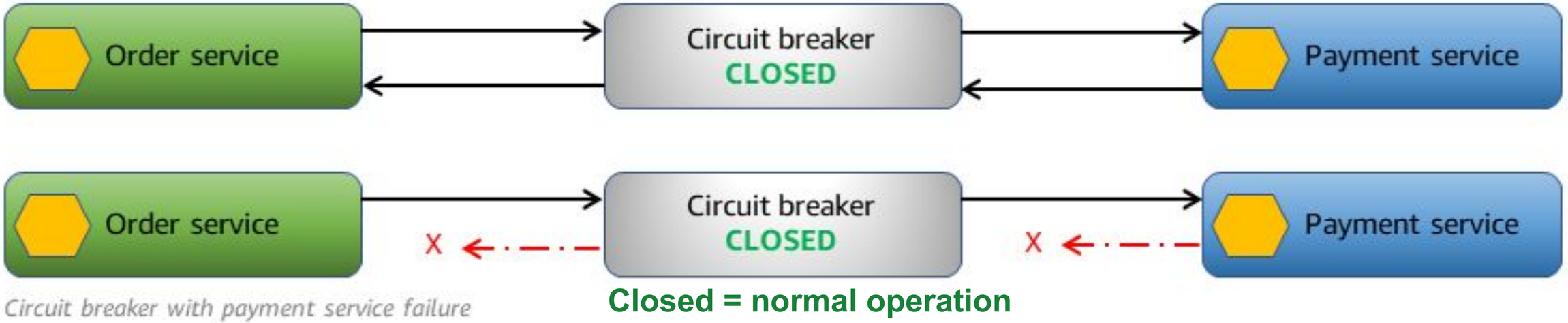
Why?

- Avoid repeated calls that are likely to fail
- Reduce wasted resources and long timeouts
- Give the failing service time to recover

After a waiting period, limited test requests can check whether the service has recovered.

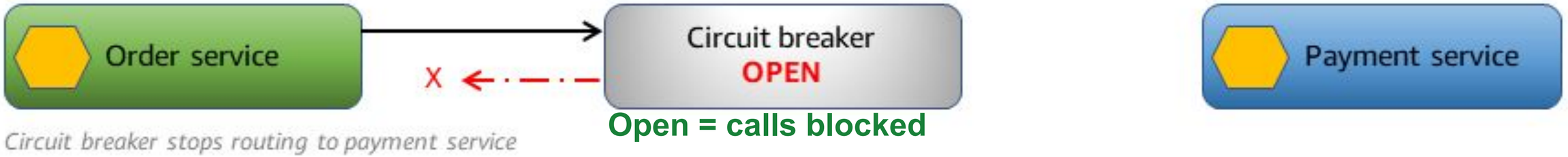
AWS: Circuit breaker pattern

Circuit breaker



Circuit breaker with payment service failure

Closed = normal operation



Circuit breaker stops routing to payment service

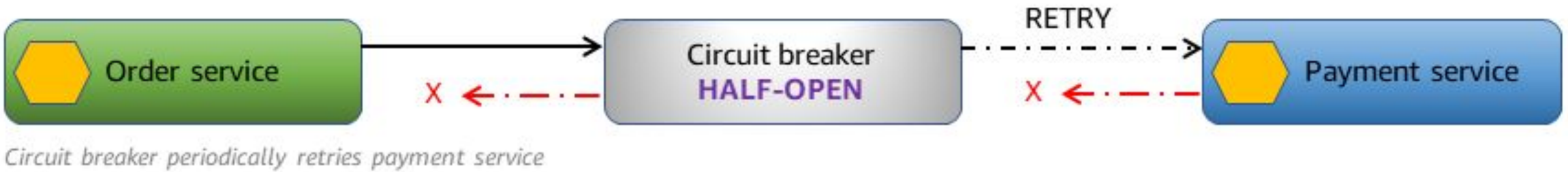
Open = calls blocked



Circuit breaker periodically retries payment service

Half-Open = test recovery

Circuit breaker



Circuit breaker periodically retries payment service



Circuit breaker with working payment service

Circuit Breaker: Advantages and Trade-offs

Advantages?

Disadvantages / Trade-offs?



Circuit Breaker: Advantages and Trade-offs

Advantages

- Prevents repeated calls to an unhealthy dependency
- Reduces wasted resources
- Can improve response time during failures
- Gives a failing service time to recover

Disadvantages / Trade-offs?

- Adds state and logic to the system
- Requires choosing failure thresholds and retry timing
- Incorrect settings can block requests too aggressively or retry too quickly
- Requires monitoring and observability

Compare the Four Patterns

Pattern	Problem	Key idea	Example
Pub/Sub	Need to decouple producers and consumers	Asynchronous messaging	Sensor events
Geode	Users geographically distributed	Multiple geographic deployments	Global web app
API Gateway	Clients must access many backend services	Single controlled entry point	Web/mobile → services
Circuit Breaker	Repeated calls to an unhealthy dependency	Temporarily stop calls	Order → payment failure

Summary

- Cloud patterns solve recurring architectural problems.
- **Pub/Sub:** decouples publishers and subscribers.
- **Geode:** distributes applications geographically.
- **API Gateway:** provides one entry point to multiple backend services.
- **Circuit Breaker:** protects systems from repeatedly calling an unhealthy dependency.
- **Choose a pattern based on the problem you need to solve.**

Practice

For each scenario, identify the most appropriate pattern or patterns and briefly explain why.

1. A mobile application needs to access Catalog, Orders, Payments, and User services without knowing each service's address.

2. The Payment Service starts timing out. The Order Service keeps retrying and is consuming more resources.

3. A global shopping site runs in several geographic regions, and an order event must be processed independently by inventory, shipping, and analytics services.

